

Part 1 — Hoisting with var

Task 1.1

Explanation:

var is hoisted, so the variable exists before its declaration. Its value is undefined until "Haridwar" is assigned.

Task 1.2

Explanation:

The local message variable is hoisted to the top of the function. It is undefined first and gets "Hello" after assignment.

Task 1.3

Explanation:

The local name variable is hoisted inside the function, so it hides the global name. Therefore, the first output is undefined.

Task 1.4

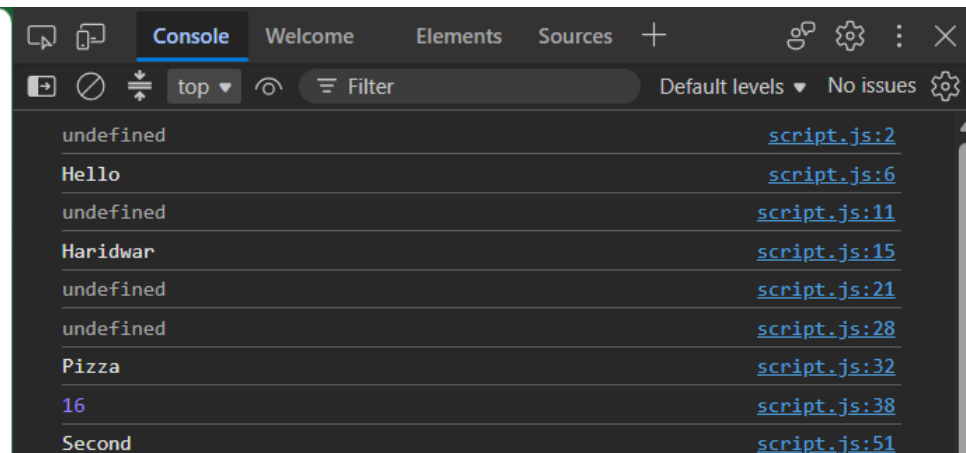
Explanation:

The variable is hoisted before its value is assigned. Therefore, the first console.log() prints undefined, and the second prints the food name.

```
<> index.html
JS script.js

1  function showMessage() {
2      console.log(message);
3
4      var message = "Hello";
5
6      console.log(message);
7  }
8
9  showMessage();
10
11 console.log(city);
12
13 var city = "Haridwar";
14
15 console.log(city);
16
17
18 var name = "global";
19
20 function test() {
21     console.log(name);
22
23     var name = "local";
24 }
25
26 test();
27
28 console.log(food);
29
30 var food = "Pizza";
31
32 console.log(food);
33
34 function square(n) {
35     return n * n;
36 }
```

JavaScript Practical Lab 6



Part 2 — Function Hoisting

Task 2.1

Explanation:

The variable sayHi is hoisted as undefined, but the function is assigned later. Calling it before assignment causes a TypeError.

Task 2.2

Explanation:

A const variable cannot be accessed before its declaration is initialized. Therefore, it gives a ReferenceError.

Task 2.3

Explanation:

Function declarations are fully hoisted, so they work before their line. Function expressions using var, let, or const cannot be called before assignment.

Task 2.4

Explanation:

Both functions have the same name. The later function declaration replaces the earlier one, so "Second" is returned

Part 3 — let, const and TDZ

Task 3.1

Explanation:

PI is accessed before its const declaration is initialized. This happens inside the **Temporal Dead Zone (TDZ)**, so a ReferenceError occurs.

Task 3.2

Explanation:

A var variable gives undefined before assignment. A let variable is in the TDZ, so accessing it before declaration gives a ReferenceError.

Task 3.3

Explanation:

undefined means the variable exists but has no value yet. ReferenceError means the variable cannot be accessed, while TypeError occurs when we try to use an inappropriate value as a function.

Part 4 — Closures

Task 4.1

Explanation:

A closure allows the inner function to remember the count variable even after the outer function finishes. counterA and counterB have separate counts.

1	script.js:85
2	script.js:86
3	script.js:87
4	script.js:88
5	script.js:89
1	script.js:91
2	script.js:92
10	script.js:106
15	script.js:107
150	script.js:139
120	script.js:140

```
index.html
JS script.js

JS script.js > makeCounter

71 //Task 4.1 - Counter//
72
73 function makeCounter() {
74     let count = 0;
75
76     return function () {
77         count++;
78         return count;
79     };
80 }
81
82 const counterA = makeCounter();
83 const counterB = makeCounter();
84
85 console.log(counterA());
86 console.log(counterA());
87 console.log(counterA());
88 console.log(counterA());
89 console.log(counterA());
90
91 console.log(counterB());
92 console.log(counterB());
93
94 //Task 4.2//
95
96 //Task 4.3 - Multiplier Factory//
97 function makeMultiplier(n) {
98     return function (x) {
99         return x * n;
100     };
101 }
102
103 const double = makeMultiplier(2);
104 const triple = makeMultiplier(3);
105
106 console.log(double(5));
```

Task 4.2

Explanation:

count is private inside makeCounter() and cannot be accessed directly outside. This shows how closures can provide data hiding.

Task 4.3

Explanation:

The returned function remembers the value of `n`. Therefore, `double` remembers 2 and `triple` remembers 3.

```
95
96 //Task 4.3 – Multiplier Factory//
97 function makeMultiplier(n) {
98     return function (x) {
99         return x * n;
100     };
101 }
102
103 const double = makeMultiplier(2);
104 const triple = makeMultiplier(3);
105
106 console.log(double(5));
107 console.log(triple(5));
108
109
```

15	script.js:107
150	script.js:139

Task 4.4**Explanation:**

`makeGreeter()` returns a function that remembers the greeting. The inner function uses that remembered value with the person's name.

Task 4.5**Explanation:**

Each cup counter has its own private count. Therefore, two different counters maintain their values separately.

Part 5 — Private Data with Closures

Task 5.1**Explanation:**

`balance` is private inside the closure, so changing `wallet.balance` does not change the actual balance.

Task 5.2**Explanation:**

The `reset()` function can use the remembered start value to restore the wallet balance to its original amount.

Task 5.3

Explanation:

The closure remembers how many attempts have been used. It allows only the specified maximum attempts and then returns "Locked!".

```
JS script.js > ...
110 //PART 5 – Private Data with Closures//
111 function createWallet(start) {
112     let balance = start;
113
114     return {
115         add(n) {
116             balance += n;
117             return balance;
118         },
119
120         spend(n) {
121             if (n > balance) {
122                 return "Insufficient balance";
123             }
124
125             balance -= n;
126             return balance;
127         },
128
129         show() {
130             return balance;
131         }
132     };
133 }
134
135 const wallet = createWallet(100);
136
137 console.log(wallet.add(50));
138 console.log(wallet.spend(30));
139 console.log(wallet.spend(500));
140 console.log(wallet.show());
141 console.log(wallet.balance);
142
```

Task 5.4

Explanation:

The diary entries are stored in a private array. Outside code cannot directly access or modify this array.

150	script.js:137
120	script.js:138
Insufficient balance	script.js:139
120	script.js:140
undefined	script.js:141
1 (2) 53 3 37	script.js:151

Part 6 — Closures in Loops

Task 6.1

Explanation:

With var, all functions share the same variable, whose final value is 3. With let, each loop iteration has its own value, so the result is [0, 1, 2].

Task 6.2

Explanation:

The var loop prints 4 three times because all callbacks share the same k. The let loop creates separate values for each iteration.

```
142
143 //PART 6 – Closures in Loops//
144
145 const withVar = [];
146
147 for (var i = 0; i < 3; i++) {
148   withVar.push(() => i);
149 }
150
151 console.log(withVar.map(f => f()));
152
153 const withLet = [];
154
155 for (let j = 0; j < 3; j++) {
156   withLet.push(() => j);
157 }
158
159 console.log(withLet.map(f => f()));
160
161 // PART 7 – Mini Project//
162
```

Task 6.3

Explanation:

Changing var to let creates a separate variable for each loop iteration. Therefore, the output becomes 1, 2, 3.

Part 7 — Smart Wallet Mini Project

createWallet()

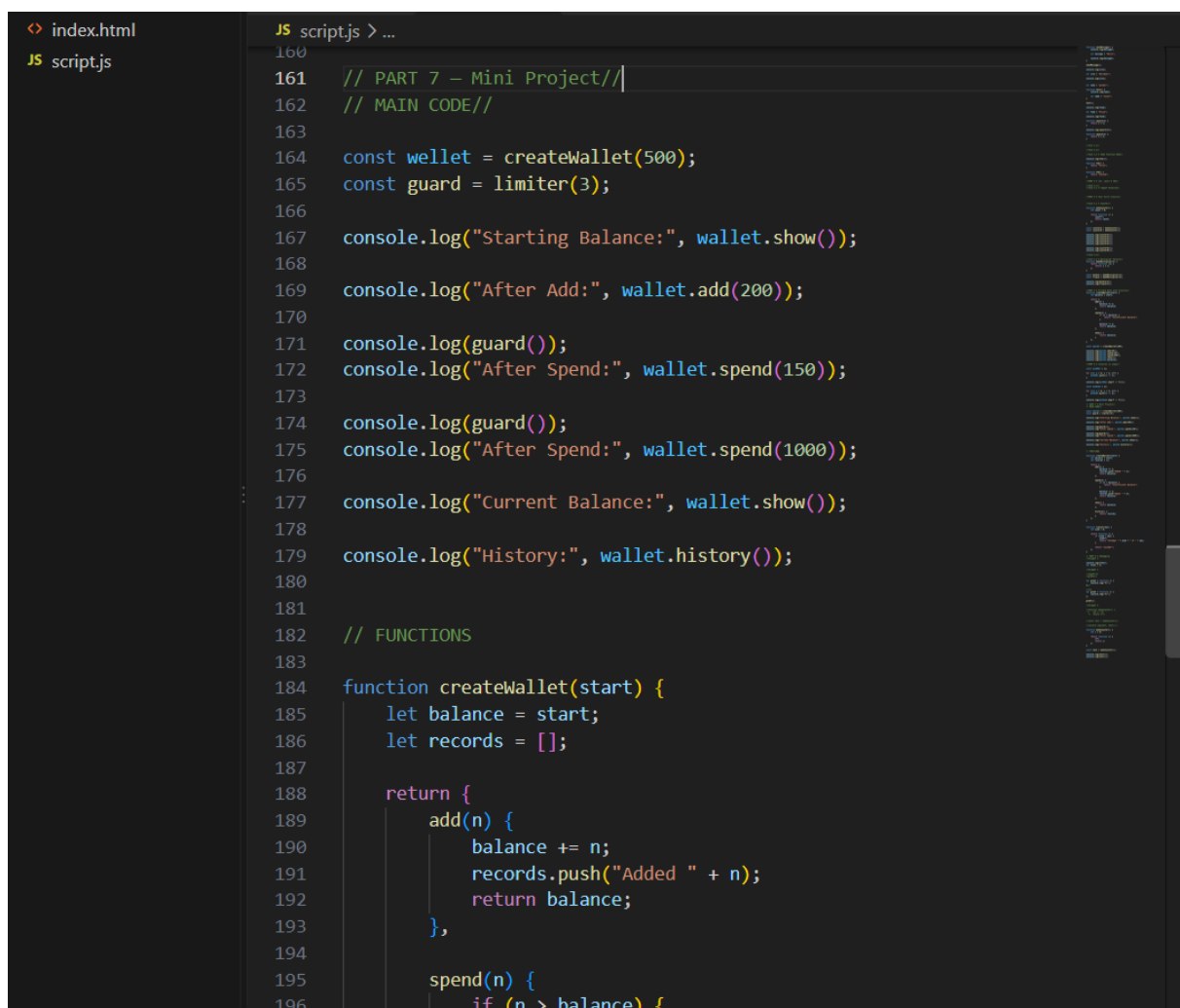
Explanation:

createWallet() creates a private wallet balance and provides functions to add, spend, show, and track transactions.

limiter()

Explanation:

limiter() uses a closure to remember the number of attempts. It allows only the maximum number of attempts and then returns "Locked!".



```
160
161 // PART 7 — Mini Project//
162 // MAIN CODE//
163
164 const wellet = createWallet(500);
165 const guard = limiter(3);
166
167 console.log("Starting Balance:", wallet.show());
168
169 console.log("After Add:", wallet.add(200));
170
171 console.log(guard());
172 console.log("After Spend:", wallet.spend(150));
173
174 console.log(guard());
175 console.log("After Spend:", wallet.spend(1000));
176
177 console.log("Current Balance:", wallet.show());
178
179 console.log("History:", wallet.history());
180
181
182 // FUNCTIONS
183
184 function createWallet(start) {
185     let balance = start;
186     let records = [];
187
188     return {
189         add(n) {
190             balance += n;
191             records.push("Added " + n);
192             return balance;
193         },
194
195         spend(n) {
196             if (n > balance) {
```

Main Code

Explanation:

The main code is placed at the top, while function declarations are placed below. Function hoisting allows the functions to be used before their definitions.

▶ (3) [3, 3, 3]

[script.js:151](#)

▶ (3) [0, 1, 2]

[script.js:159](#)

```
JS script.js > ...
182 // FUNCTIONS
183
184 function createWallet(start) {
185     let balance = start;
186     let records = [];
187
188     return {
189         add(n) {
190             balance += n;
191             records.push("Added " + n);
192             return balance;
193         },
194
195         spend(n) {
196             if (n > balance) {
197                 return "Insufficient balance";
198             }
199
200             balance -= n;
201             records.push("Spent " + n);
202             return balance;
203         },
204
205         show() {
206             return balance;
207         },
208
209         history() {
210             return records;
211         }
212     };
213 }
214
215
216 function limiter(max) {
217     let used = 0;
```

Part 8 — Debugging

Snippet 1

Explanation:

The variable is used before its value is assigned, so it prints undefined. Moving the declaration and assignment before `console.log()` fixes the problem.

Snippet 2

Explanation:

The function expression is assigned to `greet` after the function call. Moving the call after the assignment fixes the `TypeError`.

Snippet 3

Explanation:

The function returns a number instead of another function. Therefore, it cannot remember and update the counter value.

```
JS script.js > createWallet > spend
220
229 // PART 8 - Debugging
230 //Snippet 1
231
232 console.log(total);
233 var total = 5;
234
235 //Snippet 2
236
237 //TypeError
238 //greet();
239
240 var greet = function () {
241 |   console.log("Hi");
242 | };//
243
244 //fix
245 var greet = function () {
246 |   console.log("Hi");
247 | };
248
249 greet();
250
251 //Snippet 3
252
253 //function makeCounter() {
254 | //   let c = 0;
255 | //   return c++;
256 |
257 |
258 | //const next = makeCounter();
259 |
260 | //console.log(next, next);//
261 |
262 | function makeCounter() {
263 |   let c = 0;
264 |
```

```
JS script.js > createWallet > spend
244 //fix
245 var greet = function () {
246     console.log("Hi");
247 };
248
249 greet();
250
251 //Snippet 3
252
253 //function makeCounter() {
254 //    let c = 0;
255 //    return c++;
256 //}
257
258 //const next = makeCounter();
259
260 //console.log(next, next);//
261
262 function makeCounter() {
263     let c = 0;
264
265     return function () {
266         c++;
267         return c;
268     };
269 }
270
271 const next = makeCounter();
272
273 console.log(next());
274 console.log(next());
```

Snippet 4

Explanation:

The count variable is created again every time the returned function is called. Therefore, it always starts from 0 and returns 1

Starting Balance: 120	script.js:167
After Add: 320	script.js:169
Attempt 1 of 3	script.js:171
After Spend: 170	script.js:172
Attempt 2 of 3	script.js:174
After Spend: Insufficient balance	script.js:175
Current Balance: 170	script.js:177
History:	script.js:179
▶ (4) ['Added 50', 'Spent 30', 'Added 200', 'Spent 150']	
undefined	script.js:232
Hi	script.js:246
1	script.js:273
2	script.js:274

> |

